Student Budget Analysis

Salome Collante Ramirez, Andrés Ramírez, Ethan Marshall

2026-03-05

Overview

This notebook analyzes monthly rent payments reported by college students in South Florida using 2024 American Community Survey microdata (IPUMS USA). The goal is to understand what student housing budgets look like across Miami-Dade, Broward, and Palm Beach counties, and to classify students into three budget tiers (Cheap, Standard, and Luxury) based on their reported rent.

The resulting `price_category` label will serve as a join key connecting this dataset to a separate housing_data, enabling analysis of what housing options are realistically available at each budget level.

Data

About Data:

Loading Data

```
rm(list = ls())

# Working  Directory
setwd("~/Stats/Project - Housing/Final Drafts - Student Housing")

# Data
student_data = read.csv('Student_Budget.csv')

student_data$grade_level = ifelse(student_data$grade_level == 15, "Undergraduate", "Graduate")
```

Missing Values

```
colSums(is.na(student_data))
```

```
##        SERIAL       NUMPREC          puma        county location_name
##             0             0             0             0             0
##   um_distance            GQ      VEHICLES          rent           age
##             0             0             0             0             0
##   grade_level
##             0
```

Descriptive information:

```r
names(student_data)
```

```
##  [1] "SERIAL"        "NUMPREC"       "puma"          "county"
##  [5] "location_name" "um_distance"   "GQ"            "VEHICLES"
##  [9] "rent"          "age"           "grade_level"
```

```r
head(student_data)
```

```
##   SERIAL NUMPREC puma     county                 location_name um_distance GQ
## 1 303589       3 8625 Miami-Dade Coral Gables North (NEAR UM!)  VERY CLOSE  1
## 2 303619       2 1114 Miami-Dade          Miami-Dade PUMA 1114         N/A  1
## 3 303619       2 1114 Miami-Dade          Miami-Dade PUMA 1114         N/A  1
## 4 304054       3  101 Miami-Dade  North Miami Beach & Aventura         Far  1
## 5 304095       3 1902 Palm Beach          Palm Beach PUMA 1902         N/A  1
## 6 304108       2  102 Miami-Dade                   North Miami         Far  1
##   VEHICLES rent age   grade_level
## 1        1 3000  21 Undergraduate
## 2        2 1800  25      Graduate
## 3        2 1800  27      Graduate
## 4        1 1700  27      Graduate
## 5        2 1700  22 Undergraduate
## 6        2 1100  23 Undergraduate
```

```r
summary(student_data)
```

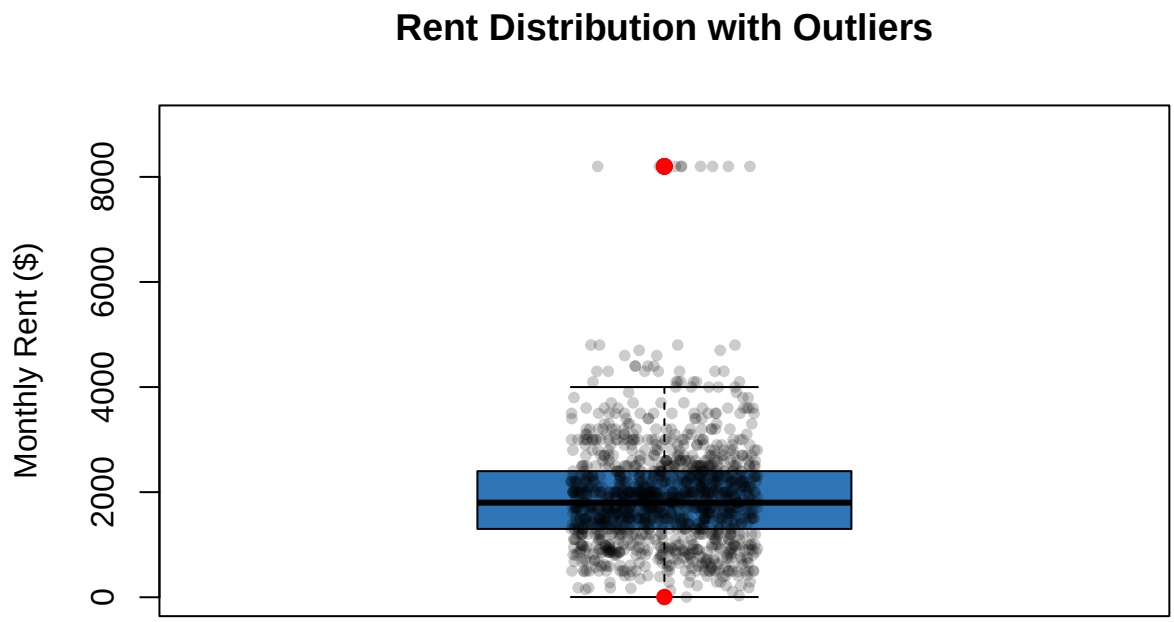
```
##      SERIAL          NUMPREC            puma         county
##  Min.   :303589   Min.   : 1.000   Min.   : 101   Length:1230
##  1st Qu.:328096   1st Qu.: 2.000   1st Qu.: 905   Class :character
##  Median :351599   Median : 3.000   Median :5705   Mode  :character
##  Mean   :349490   Mean   : 3.298   Mean   :4739
##  3rd Qu.:370714   3rd Qu.: 4.000   3rd Qu.:8610
##  Max.   :390805   Max.   :20.000   Max.   :8625
##  location_name      um_distance              GQ           VEHICLES
##  Length:1230        Length:1230        Min.   :1.000   Min.   :1.000
##  Class :character   Class :character   1st Qu.:1.000   1st Qu.:1.000
##  Mode  :character   Mode  :character   Median :1.000   Median :2.000
##                                        Mean   :1.067   Mean   :2.593
##                                        3rd Qu.:1.000   3rd Qu.:3.000
##                                        Max.   :5.000   Max.   :9.000
##       rent           age        grade_level
##  Min.   :   4   Min.   :18.00   Length:1230
##  1st Qu.:1300   1st Qu.:20.00   Class :character
##  Median :1800   Median :22.00   Mode  :character
##  Mean   :1921   Mean   :22.64
##  3rd Qu.:2400   3rd Qu.:25.00
##  Max.   :8200   Max.   :30.00
```

```r
dim(student_data)
```

```
## [1] 1230   11
```

Determining outliers

```r
# Boxplot with individual points
boxplot(student_data$rent,
        main = "Rent Distribution with Outliers",
        ylab = "Monthly Rent ($)",
        col  = "#2E75B6",
        border = "black",
        outline = FALSE,   # hides default outlier dots so we can color them manually
        ylim = c(0, 9000))   # extend to show the 8200 values

# All points
stripchart(student_data$rent,
           vertical = TRUE,
           method = "jitter",
           add = TRUE,
           pch = 20,
           col = rgb(0,0,0,0.2))

# Highlight the two extreme values in red
outlier_vals = student_data$rent[student_data$rent == 4 | student_data$rent == 8200]
stripchart(outlier_vals,
           vertical = TRUE,
           add = TRUE,
           pch = 20,
           col = "red",
           cex = 1.5)
```




```
student_data = student_data[student_data$rent > 4 & student_data$rent < 5000, ]
cat("Rows remaining:", nrow(student_data))
```

```
## Rows remaining: 1219
```

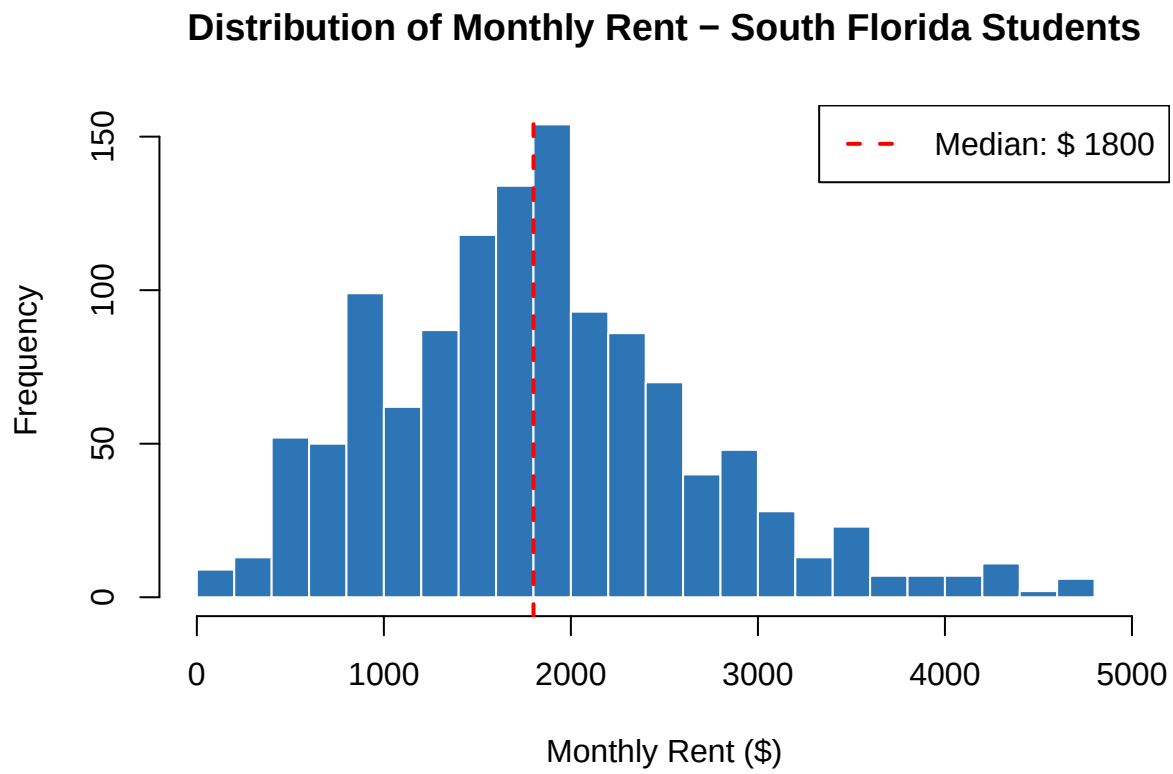
```
attach(student_data)
```

Removed 11 observations with rent values below $10 or above $5,000, as these values are clearly separated from the rest of the distribution with no observations between $4,800 and $8,200.

Descriptive Analysis:

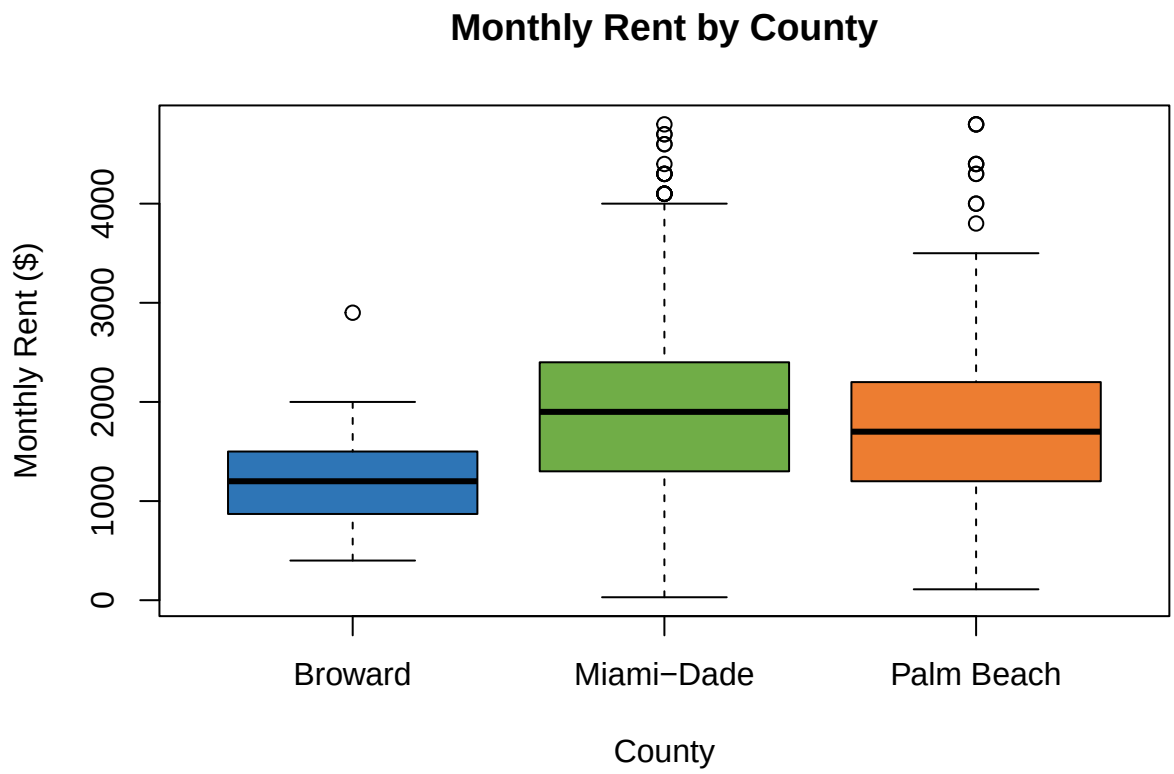
Distribution of monthly rent across all students.

```
# Histogram- Rent Dist.
hist(rent,
     main = "Distribution of Monthly Rent - South Florida Students",
     xlab = "Monthly Rent ($)",
     col  = "#2E75B6",
     border = "white",
     breaks = 30,
     xlim = c(0, 5000))
abline(v =median(rent),col = "red",lwd = 2,lty=2)
legend("topright",legend=paste("Median: $",median(rent)),col= "red",lty = 2,lwd = 2)
```



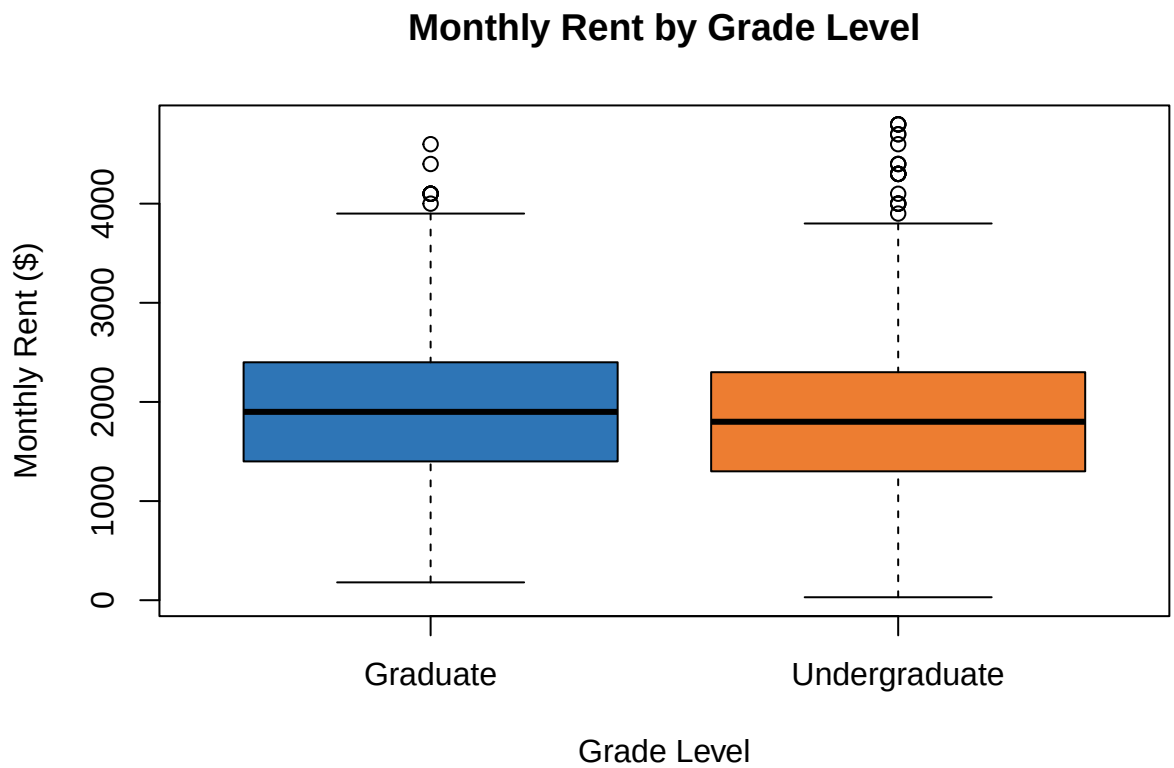
Rent by county.

```
# Box Plot- Rent by county
boxplot(rent ~ county,
        data = student_data,
        main = "Monthly Rent by County",
        xlab = "County",
        ylab = "Monthly Rent ($)",
        col  = c("#2E75B6", "#70AD47", "#ED7D31"))
```



Rent by grade level (Undergraduate | Graduate).

```
# Boxplot- Rent by grade level
boxplot(rent ~ grade_level,
        data = student_data,
        main = "Monthly Rent by Grade Level",
        xlab = "Grade Level",
        ylab = "Monthly Rent ($)",
        col  = c("#2E75B6", "#ED7D31"))
```



Summary statistics by (1)county, (2)grade level.

```
tapply(rent, county, summary)
```

```
## $Broward
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     400     870    1200    1248    1500    2900
##
## $`Miami-Dade`
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      30    1300    1900    1896    2400    4800
##
## $`Palm Beach`
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     110    1200    1700    1817    2200    4800
```

```
tapply(rent, grade_level, summary)
```

```
## $Graduate
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     180    1400    1900    1917    2400    4600
##
## $Undergraduate
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      30    1300    1800    1857    2300    4800
```

Budget Tier Assignment:

Students are classified into three budget tiers based on their monthly rent.

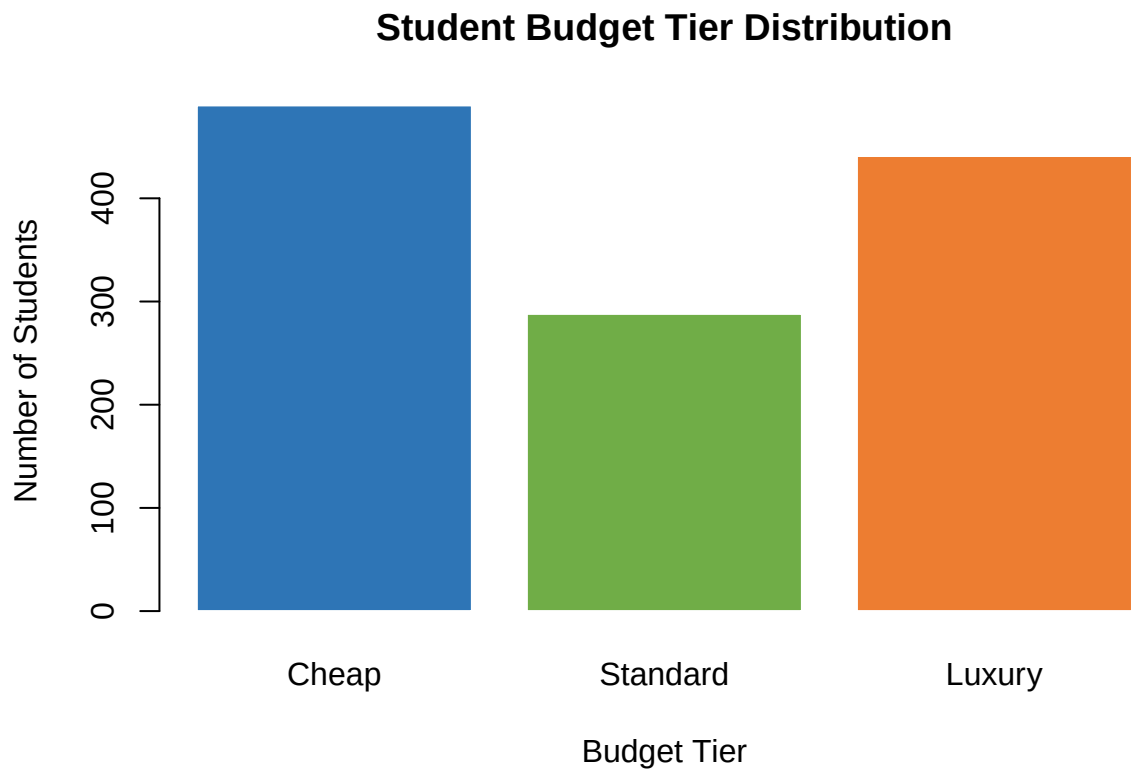
Tier	Rent Range	Description
Cheap	$0 – $1,600	Shared housing, budget apartments
Standard	$1,601 – $2,000	Mid-range apartments
Luxury	$2,001+	High-end or solo units

```r
student_data$price_category = cut(
  student_data$rent,
  breaks = c(0, 1600, 2000, 5000),
  labels = c("Cheap", "Standard", "Luxury")
)

summary(student_data$price_category)
```

```
##    Cheap Standard   Luxury
##      490      288      441
```

Distribution of budget tiers.

```r
counts = table(student_data$price_category)
barplot(counts,
        main = "Student Budget Tier Distribution",
        xlab = "Budget Tier",
        ylab = "Number of Students",
        col  = c("#2E75B6", "#70AD47", "#ED7D31"),
        border = "white")
```



Tier breakdown by county.

```
table(student_data$county,student_data$price_category)
```

```
##
##                Cheap Standard Luxury
##   Broward         13        3      1
##   Miami-Dade     359      229    362
##   Palm Beach     118       56     78
```

Tier breakdown by grade level.

```
table(student_data$grade_level,student_data$price_category)
```

```
##
##                   Cheap Standard Luxury
##   Graduate           98       75    106
##   Undergraduate     392      213    335
```

Data Preparation for Modeling:

```
# Using age, and county as predictors
data_class = student_data[, c("rent","age","county","price_category")]
```

```r
data_class$county =  as.factor(data_class$county)
data_class$price_category =  as.factor(data_class$price_category)

data_class = na.omit(data_class)

dim(data_class)
```

```
## [1] 1219    4
```

Train / Test Split:

```r
set.seed(1)

# 80% of observations for training
num_samples = nrow(data_class)
train = sample(num_samples,num_samples * 0.8)

dim(data_class[train, ])
```

```
## [1] 975   4
```

```r
dim(data_class[-train, ])
```

```
## [1] 244   4
```

```r
x = model.matrix(price_category ~.,data=data_class)[, -1]
train.x = x[train, ]
test.x = x[-train, ]

# Standardize variables (mean=0, sd=1)
train.x = scale(train.x)
test.x = scale(test.x,
               center = attr(train.x,"scaled:center"),
               scale  = attr(train.x,"scaled:scale"))

# Replace NaN from zero-variance columns with 0
train.x[is.nan(train.x)] = 0
test.x[is.nan(test.x)] = 0

train.y = data_class$price_category[train]
test.y = data_class$price_category[-train]
```

10-Fold Cross Validation:

```r
library(class)
library(MASS)
library(randomForest)
```

```r
library(pROC)

k = 10
set.seed(1)

# Vector that allocates each observation to one of k = 10 folds
folds = sample(1:k, nrow(data_class[train, ]), replace = TRUE)

# Matrix for results
cv.errors = matrix(NA,k,3,dimnames=list(NULL, c("LDA", "KNN", "RF")))


for (j in 1:k) {

  # KNN
  cv.train.x = train.x[folds != j, ]
  cv.test.x  = train.x[folds == j, ]
  cv.train.y = train.y[folds != j]
  cv.test.y  = train.y[folds == j]

  knn.cv = knn(cv.train.x,cv.test.x,cl=cv.train.y,k = 5)
  cv.errors[j, "KNN"]=mean(knn.cv==cv.test.y)

  # LDA
  cv.train.df = data_class[train, ][folds != j, ]
  cv.test.df  = data_class[train, ][folds == j, ]

  lda.cv = lda(price_category ~.,data=cv.train.df)
  lda.cv.pred = predict(lda.cv,cv.test.df)$class
  cv.errors[j,"LDA"] = mean(lda.cv.pred==cv.test.y)

  # Random Forest
  rf.cv = randomForest(price_category ~.,data=cv.train.df,ntree=500)
  rf.cv.pred = predict(rf.cv,newdata=cv.test.df)
  cv.errors[j,"RF"] = mean(rf.cv.pred==cv.test.y)
}

# Average over columns
mean.cv.errors = apply(cv.errors,2,mean)
mean.cv.errors
```

```
##       LDA       KNN        RF
## 0.9223660 0.9428132 1.0000000
```

KNN:

```r
# KNN = 1
knn.pred = knn(train.x, test.x, train.y, k=1)
train.knn.pred = knn(train.x, train.x, train.y, k=1)

# Confusion matrix
```

```r
conf_matrix = table(knn.pred,test.y)
print(conf_matrix)
```

```
##           test.y
## knn.pred   Cheap Standard Luxury
##   Cheap       92        4      0
##   Standard     0       64      6
##   Luxury       0        0     78
```

```r
# Accuracy score
knn_train_acc = mean(train.knn.pred == train.y)
knn_test_acc = mean(knn.pred == test.y)

# AUC score
knn.pred.auc = knn(train.x, test.x, train.y, k=1, prob=TRUE)
knn.prob.auc = attr(knn.pred.auc, "prob")
knn.test.roc = multiclass.roc(test.y, knn.prob.auc)
```

```
## Setting direction: controls < cases
## Setting direction: controls < cases
## Setting direction: controls < cases
```

```r
knn_test_auc = auc(knn.test.roc)

cat('Train accuracy: ', knn_train_acc, "\n")
```

```
## Train accuracy:  1
```

```r
cat('Test accuracy: ', knn_test_acc, "\n")
```

```
## Test accuracy:  0.9590164
```

```r
cat('Test AUC: ', knn_test_auc, "\n")
```

```
## Test AUC:  0.5
```

```r
# K = 11
knn.11.pred = knn(train.x, test.x, train.y, k=11)
train.11.pred = knn(train.x, train.x, train.y, k=11)

# Confusion matrix
conf_matrix = table(knn.11.pred,test.y)
print(conf_matrix)
```

```
##            test.y
## knn.11.pred Cheap Standard Luxury
##    Cheap       92        2      0
##    Standard     0       66      5
##    Luxury       0        0     79
```

```r
# Accuracy score
knn11_train_acc = mean(train.11.pred == train.y)
knn11_test_acc = mean(knn.11.pred == test.y)

# AUC score
knn11.pred.auc = knn(train.x, test.x, train.y, k=11, prob=TRUE)
knn11.prob.auc = attr(knn11.pred.auc, "prob")
knn11.test.roc = multiclass.roc(test.y, knn11.prob.auc)
```

```
## Setting direction: controls > cases
## Setting direction: controls < cases
## Setting direction: controls < cases
```

```r
knn11_test_auc = auc(knn11.test.roc)

cat('Train k = 11 accuracy: ', knn11_train_acc, "\n")
```

```
## Train k = 11 accuracy:  0.9671795
```

```r
cat('Test k = 11 accuracy: ', knn11_test_acc, "\n")
```

```
## Test k = 11 accuracy:  0.9713115
```

```r
cat('Test k = 11 AUC: ', knn11_test_auc, "\n")
```

```
## Test k = 11 AUC:  0.6905954
```

LDA:

```r
lda.fit = lda(price_category ~.,data=data_class,subset=train)
lda.pred = predict(lda.fit, data_class[-train, ])
train.pred = predict(lda.fit, data_class[train, ])

# Confusion matrix
conf_matrix = table(lda.pred$class, test.y)
print(conf_matrix)
```

```
##           test.y
##            Cheap Standard Luxury
##   Cheap       87        4      0
##   Standard     5       64     16
##   Luxury       0        0     68
```

```r
# Accuracy score
lda_train_acc = mean(train.pred$class == train.y)
lda_test_acc = mean(lda.pred$class == test.y)

# AUC score
```

```r
lda_test_roc = multiclass.roc(test.y, lda.pred$posterior)
lda_test_auc = auc(lda_test_roc)

cat('Train accuracy: ', lda_train_acc, "\n")
```

```
## Train accuracy:  0.9241026
```

```r
cat('Test accuracy: ', lda_test_acc, "\n")
```

```
## Test accuracy:  0.897541
```

```r
cat('Test AUC: ', lda_test_auc)
```

```
## Test AUC:  0.9917857
```

Random Forest:

```r
set.seed(1)
rf.fit = randomForest(price_category ~.,data=data_class,subset=train,ntree=500,importance=T)
yhat.rf = predict(rf.fit, newdata=data_class[-train, ])
train.rf = predict(rf.fit, newdata=data_class[train, ])

# Confusion matrix
conf_matrix = table(yhat.rf, test.y)
print(conf_matrix)
```

```
##           test.y
## yhat.rf    Cheap Standard Luxury
##   Cheap       92        0      0
##   Standard     0       68      0
##   Luxury       0        0     84
```

```r
# Accuracy score
rf_train_acc = mean(train.rf == train.y)
rf_test_acc = mean(yhat.rf == test.y)

# AUC score
yhat.rf.auc = predict(rf.fit, newdata=data_class[-train, ], type='prob')
rf_test_roc = multiclass.roc(test.y, yhat.rf.auc)
rf_test_auc = auc(rf_test_roc)

cat('Train accuracy: ', rf_train_acc, "\n")
```

```
## Train accuracy:  1
```

```r
cat('Test accuracy: ', rf_test_acc, "\n")
```

```
## Test accuracy:  1
```

```r
cat('AUC score:', rf_test_auc)
```

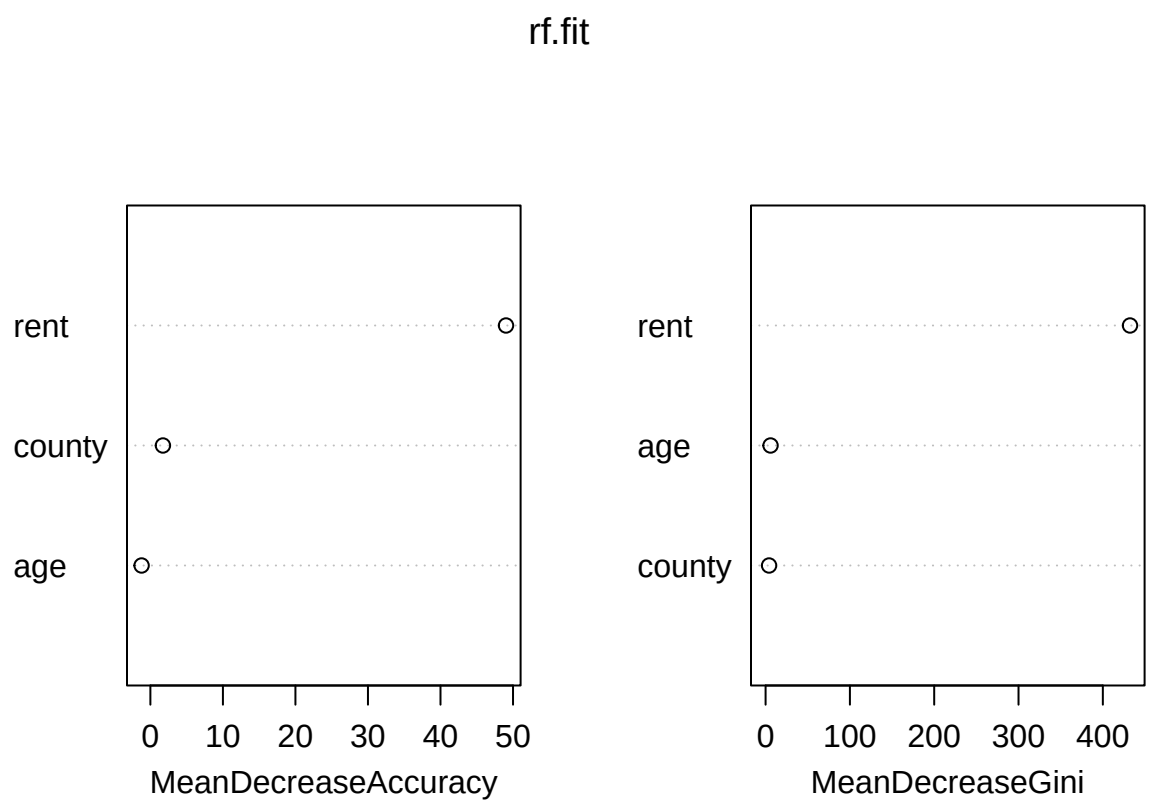
```
## AUC score: 1
```

Variable Importance

```r
importance(rf.fit)
```

```
##            Cheap   Standard    Luxury MeanDecreaseAccuracy MeanDecreaseGini
## rent   53.570674 33.3731582 48.815495            49.035331       432.236711
## age    -2.495925  0.8052657 -0.987361            -1.213541         5.908635
## county  2.582278 -3.1431261  2.987657             1.730869         4.155545
```

```r
varImpPlot(rf.fit)
```



Model Comparison Summary:

```r
# Model names
models = c("LDA", "KNN_1", "KNN_11", "RF")

# Training accuracy
```

```r
train_acc = c(lda_train_acc, knn_train_acc, knn11_train_acc, rf_train_acc)

# Test accuracy
test_acc = c(lda_test_acc, knn_test_acc, knn11_test_acc, rf_test_acc)

# Mean 10-fold CV
fold10 = c(mean.cv.errors["LDA"], mean.cv.errors["KNN"], mean.cv.errors["KNN"], mean.cv.errors["RF"])

# AUC scores
auc_val = c(lda_test_auc, knn_test_auc, knn11_test_auc, rf_test_auc)

summary_table = data.frame(
  Model = models,
  Train_Acc = train_acc,
  Test_Acc = test_acc,
  Fold10 = fold10,
  AUC = auc_val
)

print(summary_table)
```

```
##    Model Train_Acc  Test_Acc    Fold10       AUC
## 1    LDA 0.9241026 0.8975410 0.9223660 0.9917857
## 2  KNN_1 1.0000000 0.9590164 0.9428132 0.5000000
## 3 KNN_11 0.9671795 0.9713115 0.9428132 0.6905954
## 4     RF 1.0000000 1.0000000 1.0000000 1.0000000
```

Our ROC results were:

```r
# Cheap vs all ROC
plot.roc(roc(test.y, lda.pred$posterior[,1]), col="blue", main="CHEAP vs all ROC")
```

```
## Setting levels: control = Cheap, case = Standard
```

```
## Setting direction: controls > cases
```

```r
lines.roc(roc(test.y, knn.test.roc$rocs[[1]]$original.predictor), col="red")
```

```
## Setting levels: control = Cheap, case = Standard
```

```
## Setting direction: controls < cases
```

```r
lines.roc(roc(test.y, knn11.test.roc$rocs[[1]]$original.predictor), col="yellow")
```

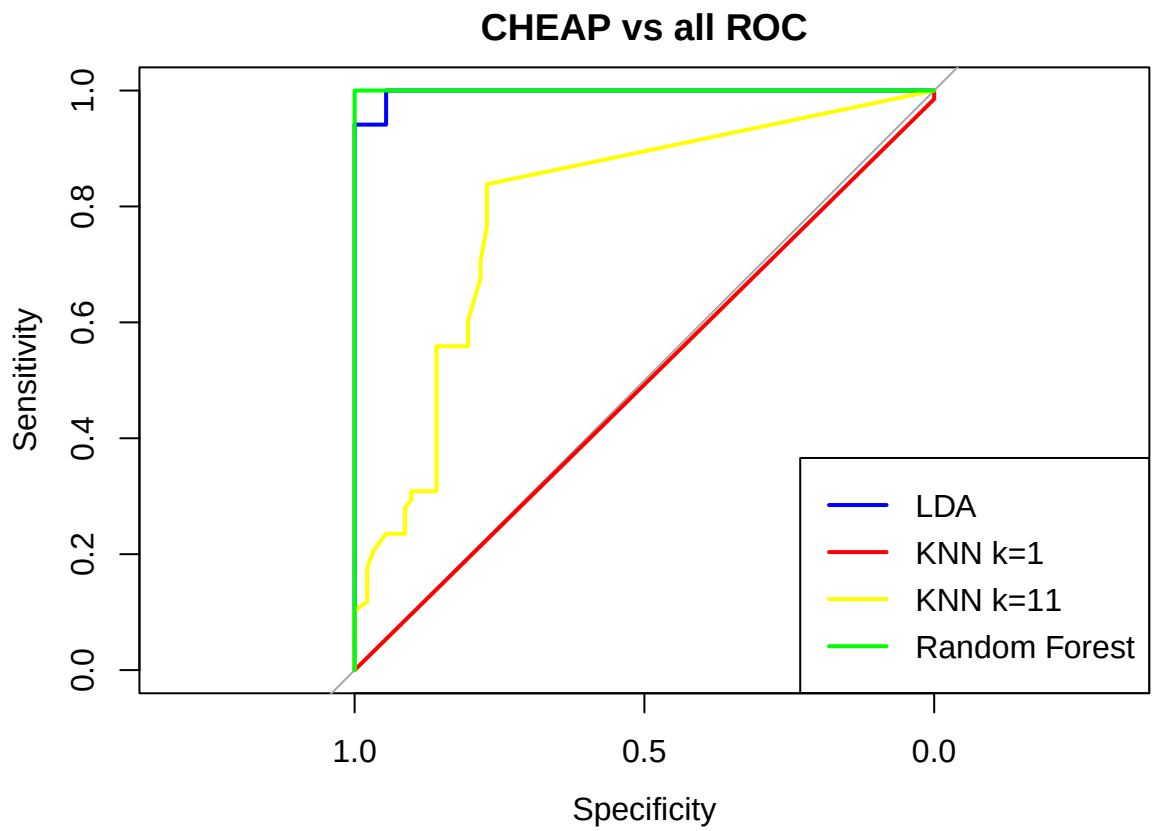
```
## Setting levels: control = Cheap, case = Standard
```

```
## Setting direction: controls > cases
```

```r
lines.roc(roc(test.y, yhat.rf.auc[,1]), col="green")
```

```
## Setting levels: control = Cheap, case = Standard
## Setting direction: controls > cases
```

```r
legend("bottomright", legend=c("LDA","KNN k=1", "KNN k=11","Random Forest"),
       col=c("blue","red","yellow","green"), lwd=2)
```




```r
# Standard vs all ROC
plot.roc(roc(test.y, lda.pred$posterior[,2]), col="blue", main="Standard vs all ROC")
```

```
## Setting levels: control = Cheap, case = Standard

## Setting direction: controls < cases
```

```r
lines.roc(roc(test.y, knn.test.roc$rocs[[2]]$original.predictor), col="red")
```

```
## Setting levels: control = Cheap, case = Standard
## Setting direction: controls < cases
```

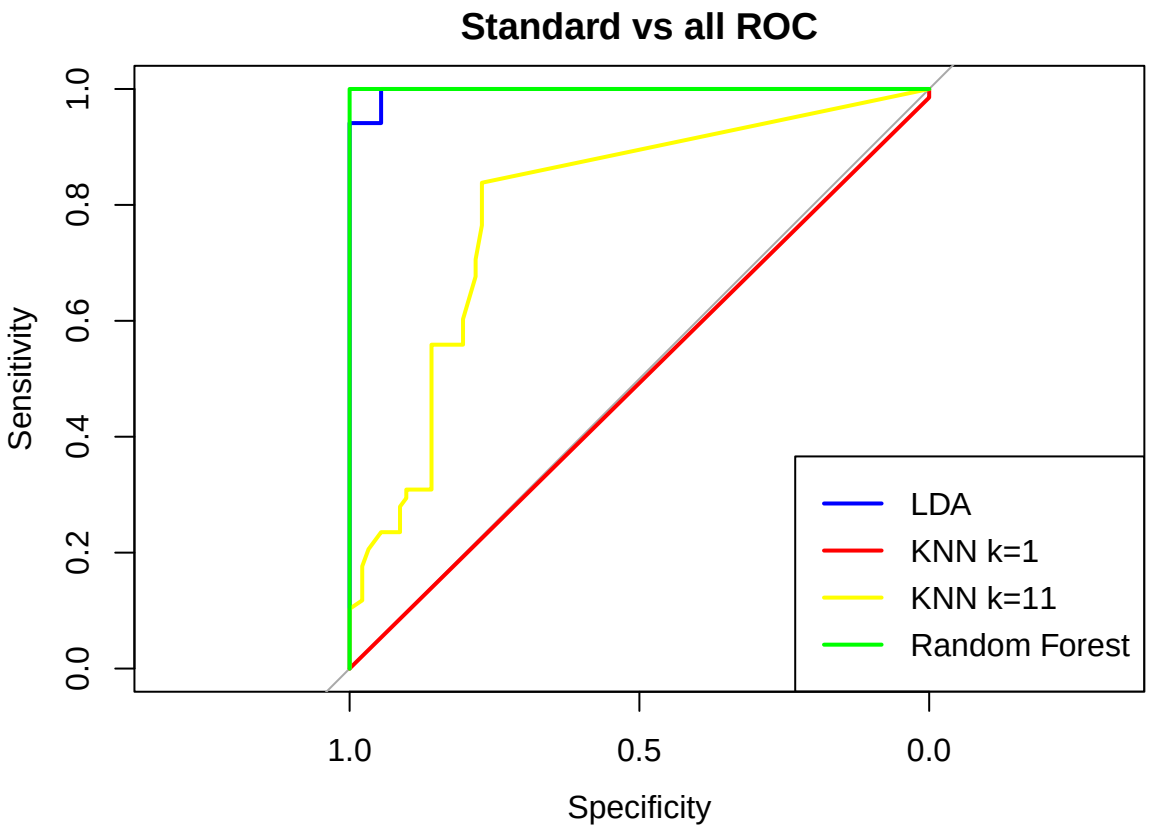
```r
lines.roc(roc(test.y, knn11.test.roc$rocs[[2]]$original.predictor), col="yellow")
```

```
## Setting levels: control = Cheap, case = Standard

## Setting direction: controls > cases
```

```
lines.roc(roc(test.y, yhat.rf.auc[,2]), col="green")
```

```
## Setting levels: control = Cheap, case = Standard

## Setting direction: controls < cases
```

```
legend("bottomright", legend=c("LDA","KNN k=1", "KNN k=11","Random Forest"),
       col=c("blue","red","yellow","green"), lwd=2)
```




```
# Luxury vs all ROC
plot.roc(roc(test.y, lda.pred$posterior[,3]), col="blue", main="Luxury vs all ROC")
```

```
## Setting levels: control = Cheap, case = Standard
## Setting direction: controls < cases
```

```
lines.roc(roc(test.y, knn.test.roc$rocs[[3]]$original.predictor), col="red")
```

```
## Setting levels: control = Cheap, case = Standard
## Setting direction: controls < cases
```

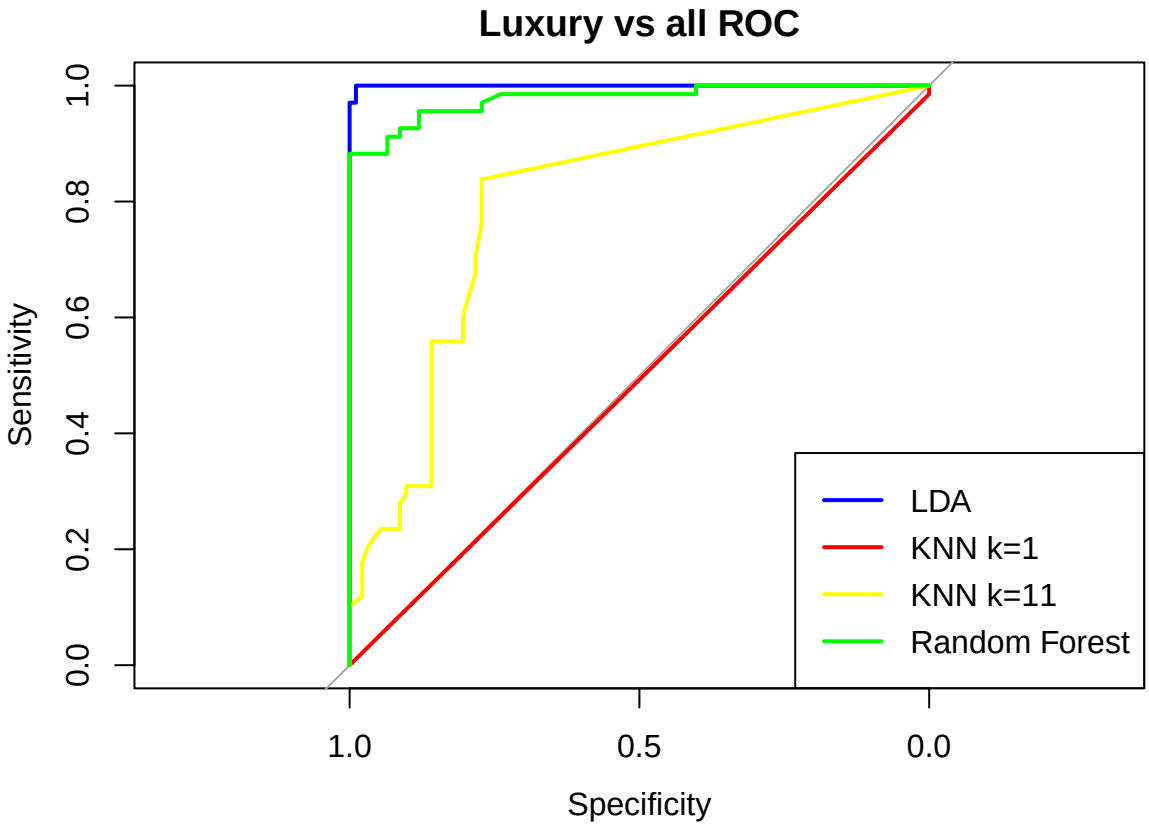
```r
lines.roc(roc(test.y, knn11.test.roc$rocs[[3]]$original.predictor), col="yellow")
```

```
## Setting levels: control = Cheap, case = Standard
```

```
## Setting direction: controls > cases
```

```r
lines.roc(roc(test.y, yhat.rf.auc[,3]), col="green")
```

```
## Setting levels: control = Cheap, case = Standard
```

```
## Setting direction: controls < cases
```

```r
legend("bottomright", legend=c("LDA","KNN k=1", "KNN k=11","Random Forest"),
       col=c("blue","red","yellow","green"), lwd=2)
```